

Library Management System — Requirement Analysis

1. Member Management

- Register members with name, phone, email, password, and membership tier
- Email must be valid format; phone must be exactly 10 digits
- Email and phone are unique — enforced at both service layer and DB (unique indexes)
- Deactivation is blocked if the member has unreturned books or unpaid fines
- Members can self-service: edit profile (name, phone, email, tier) and change password
- Admins can edit any member's profile from the admin panel
- Uniqueness re-validated on update, excluding the member's own current values

2. Book Management

- Catalog tracks logical books (title, author, ISBN, year, category) and physical copies separately
- Admin can add, edit, and delete books; delete is blocked if any copy is currently borrowed
- Adding a copy shows the book title first — no more blind ID entry
- Admin can view all copies of a book with status breakdown (Available / Borrowed / Out of service)
- Copy lifecycle: Available → Borrowed → Available (normal) or → Unavailable (damaged return)
- ISBN uniqueness and accession number uniqueness enforced at DB level

3. Borrowing Workflow

All validation and writes run inside `proc_borrow_book` — atomic, no partial state.

Checks in order:

1. Member is active
2. Unpaid fines ≤ ₹500
3. Active borrowing count < tier limit
4. Copy status is Available
5. Member doesn't already have this book borrowed

On success: inserts Borrowing record, marks copy Borrowed, sets due date = borrow date + tier's MaxBorrowDays.

4. Return Workflow

Handled by `proc_return_book(borrowing_id, is_damaged)` — atomic.

- **On-time, undamaged:** copy → Available, no fine
- **Overdue:** fine = overdue days × ₹10, copy → Available
- **Damaged:** flat ₹500 fine, copy → Unavailable + Condition = Damaged
- **Overdue + damaged:** single combined fine with breakdown in result message

Member is asked about damage condition in the UI before the call is made.

5. Fine Management

- Fines are created automatically by `proc_return_book`
- Full payment only — no installments
- `IsPaid` flips to true and `PaidDate` is set when payment is made
- `calculate_member_fine(member_id)` — PostgreSQL function, returns total outstanding balance
- Called inside `proc_borrow_book` to enforce the ₹500 borrowing block
- Members can only pay their own fines (ownership validated in UI)

6. Reporting

All six reports backed by PostgreSQL functions — no in-memory LINQ joins.

Report	Function
Books currently borrowed	<code>fn_get_currently_borrowed_books()</code>

Overdue books Report	fn_get_overdue_books() Function
Members with pending fines	fn_get_members_with_pending_fines()
Most borrowed books	fn_get_most_borrowed_books()
Available books by category	fn_get_available_books_by_category()
Member borrowing history	fn_get_member_borrowing_history(id)

C# side uses a two-query pattern: function returns IDs → EF loads full entities with navigation properties.

7. Business Rules

Rule	Detail
Tiered borrowing limits	Basic: 2/7d · Student: 3/10d · Premium: 5/15d
Fine block	Unpaid balance > ₹500 → no new borrowings
Duplicate borrow prevention	Can't borrow two copies of the same book simultaneously
Inactive account	Deactivated members can't borrow
Deactivation guard	Blocked if member has unreturned books or unpaid fines
Damage fine	Damaged return → ₹500 flat fine + copy retired from circulation
Book deletion guard	Blocked if any copy is currently borrowed
Atomic transactions	Borrow and return run in PostgreSQL procedures with own COMMIT/ROLLBACK

8. Domain Model

Entity	Role
Member	Library member — holds profile, tier, active status, admin flag
MembershipType	Tier definition — max books and max days. Seeded: Basic, Student, Premium
BookCategory	Classification label — Fiction, Non-Fiction, Science, History, Biography
Book	Logical publication — title, author, ISBN, year, category
BookCopy	Physical copy — accession number, condition, status
Borrowing	Transaction record — links member to copy for a period
Fine	Penalty record — amount, amount paid, issued date, paid date

Key relationships:

- Book (1) → (N) BookCopy — cascade delete
- Member (1) → (N) Borrowing — restrict delete
- Member (1) → (N) Fine — restrict delete
- Borrowing (1) → (1) Fine — restrict delete

9. Database Design

- **3NF** — `Book` and `BookCopy` are separate to avoid duplicating title/author per copy
- **Unique indexes** — `Members.Email`, `Members.Phone`, `BookCopies.AccessionNumber`, `Books.ISBN` (partial, when provided)
- **EF Core mirrors DB constraints** — `HasIndex(...).IsUnique()` in `LibraryContext`
- **Status as strings** — `Available` / `Borrowed` / `Unavailable` / `Damaged` — readable, small value set
- **Cascade vs Restrict** — `Book→Copy` is cascade; everything else is restrict to preserve history

10. Stored Procedures vs Functions

`procedures.sql` — write operations, own their transaction:

- `proc_borrow_book(member_id, copy_id)` — full borrow flow, OUT result message
- `proc_return_book(borrowing_id, is_damaged)` — full return flow, OUT result message

Using `PROCEDURE` (not `FUNCTION`) because they need `COMMIT` / `ROLLBACK` inside the body. A function runs in the caller's transaction; a procedure owns its own.

`functions.sql` — pure reads, no side effects:

- `calculate_member_fine(member_id)` — scalar, total outstanding fine
- `get_member_borrowing_summary(member_id)` — active/returned/overdue counts + fine total
- Six report functions (see section 6)

11. Architecture

Four-layer separation:

1. **UI (Console)** — input/output, basic type validation, menu routing. No business logic.
2. **BLL (Services)** — business rules, guards, validation, orchestration.
3. **DAL (Repositories)** — all DB interaction. EF Core LINQ for reads/simple writes; `ExecuteSqlRaw` for procedure calls.
4. **Infrastructure (EF Core + PostgreSQL)** — ORM config, unique indexes, SQL objects.

12. Validation Layers

Layer	What it catches
UI	Type parsing, non-empty checks, ownership (does this fine/borrowing belong to me?)
BLL	Email/phone format, uniqueness, membership type existence, deactivation guards, password verification
DB	FK constraints, unique indexes, procedure-level business rule checks

13. Exception Hierarchy

All extend `LibraryException` :

- `InvalidEmailException` / `InvalidPhoneException` — format failures
- `DuplicateEmailException` / `DuplicatePhoneException` — uniqueness violations
- `InvalidMembershipTypeException` — unknown tier ID
- `MemberNotFoundException` / `BookNotFoundException` — entity not found
- `MemberInactiveException` — suspended account tried to act
- `InvalidCredentialsException` — wrong password
- `FineNotFoundException` — fine ID invalid or already paid
- `MemberHasActiveBorrowingsException` — deactivation blocked by unreturned books
- `MemberHasUnpaidFinesException` — deactivation blocked by outstanding fines

14. Security

- **Passwords** — SHA-256 hex hash, never stored or logged as plaintext
- **Ownership checks** — members can only pay their own fines and return their own borrowings
- **Atomic writes** — procedures guarantee no partial state on crash
- **Duplicate prevention** — app-level check first, DB unique index as fallback
- **Inventory guards** — books can't be deleted while borrowed; members can't be deactivated while holding books or owing fines

